

Project Overview

This project is designed to integrate a locally-hosted, open-source Language Model (LLM) into a fantasy RPG duel game, which is represented as a game of chess. The LLM generates dynamic narrations and visual effects for each chess move, enhancing the gaming experience by providing engaging descriptions and animations. If the LLM is unavailable or fails to generate valid output, the system falls back to predefined narrations and effects to ensure the game continues smoothly.

Architecture

The architecture of this project revolves around a single function `get_narration`` that interacts with the LLM to generate narrations and effects for chess moves. It includes fallback mechanisms to handle errors gracefully. The system categorizes moves based on their characteristics (e.g., checkmate, capture) and uses these categories to determine appropriate fallback narrations and effects.

Key Components

- **LLM Interaction**: The core component that communicates with the locally-hosted LLM to generate narrations and effects.
- **Fallback Mechanisms**: Provides default narrations and effects when the LLM is unreachable or returns invalid data.
- **Move Categorization**: Determines the type of move to tailor the narration and effect accordingly.

Main Functions

- **get_narration(move_info, timeout=8)**: Generates a narration and effect for a given chess move. It handles exceptions and uses fallbacks if necessary.
- **_fallback_narration(move_info)**: Provides a default narration based on the type of move.
- **_fallback_effect(move_info)**: Selects a default effect from a predefined list based on the move category.
- **build_prompt(move_info)**: Constructs a prompt string to be sent to the LLM, containing the move data and instructions for generating the response.
- **_category(move_info)**: Categorizes the move based on its attributes (e.g., checkmate, captured).

Data Flow

1. **Chess Move Input**: The game sends move information to the `get_narration`` function.
2. **Prompt Construction**: The `build_prompt`` function constructs a prompt using the move information.
3. **LLM Communication**: The constructed prompt is sent to the LLM via a POST request.
4. **Response Handling**: The response from the LLM is parsed. If successful, the narration and effect are extracted; otherwise, fallbacks are used.
5. **Output**: The function returns a dictionary containing the narration, effect, and source of the data (either 'llm' or 'fallback').

Setup Instructions

1. **Install Dependencies**:

- Ensure Python is installed on your system.
- Install the required Python packages using pip:

```
```bash
pip install requests
```
```

2. **Configure Environment Variables**:

- Set the `OLLAMA\_URL` environment variable to point to your locally-hosted LLM server.
- Define the `MODEL\_NAME` environment variable with the name of the model you intend to use.

3. **Run the Application**:

- Integrate the `get\_narration` function into your chess game application to generate narrations and effects for each move.

Future Improvements

- **Enhanced Error Handling**: Improve error handling to provide more detailed logs and user feedback.
- **Customizable Fallbacks**: Allow users to customize fallback narrations and effects based on their preferences.
- **Model Selection UI**: Implement a user interface to select different models hosted on Ollama.
- **Performance Optimization**: Optimize the data flow and processing to reduce latency and improve performance.
- **Testing Framework**: Develop a comprehensive testing framework to ensure reliability and accuracy of the generated narrations and effects.

API Endpoints

- **POST `/generate`**: This endpoint is not explicitly defined in the provided code but represents the interaction with the LLM. The `get\_narration` function sends a POST request to `OLLAMA\_URL` with the move data and options.

Folder Structure

The provided repository context does not include a folder structure. Assuming a typical Python project structure, it might look like this:

```
```
chess_rpg/
├── main.py # Contains the main functions and logic
├── utils.py # Utility functions like _fallback_narration, _fallback_effect, _category
├── config.py # Configuration settings such as OLLAMA_URL, MODEL_NAME, PIECE_NAMES,
VALID_EFFECTS, FALLBACK_EFFECTS
├── tests/ # Directory for test cases
└── test_main.py # Test file for main functionalities
```
```